ML MODEL CLASSIFICATION

Project Documentation

From a Synthetic Dataset to a Live Machine Learning Detector

Deployed on the Local Sentinel Guard Server

PROJECT	Sentinel Guard - IoT Anomaly Detection
DATASET	IOT-temp-colombo.csv (synthetic)
MODEL	Gradient-Boosted Classifier (XGBoost)
DEPLOYED AS	best_model.pkl and scaler.pkl
AUDIENCE	End Users and Non-ML Readers

Table of Contents

1. Project Overview

This section provides a high-level overview of the system, as a clear understanding of the overall architecture facilitates comprehension of the more detailed technical content that follows.

The system is centred on an ESP32 device, a compact embedded sensor board, which continuously measures ambient temperature and humidity. At regular intervals, the device transmits these readings to a server. The server is tasked with evaluating each incoming reading and determining whether it represents a normal observation or one that needs further review.

Readings that deviate from expected behaviour are referred to as anomalies. An anomaly may manifest as a sudden temperature spike, a sensor that has become fixed on a single unchanging value, or a reading that appears individually credible yet is different with the expected conditions for a given time of day. Some anomalies arise from benign sensor malfunctions, others may represent purposeful tampering with the device or injection of fabricated data. The system does not attempt to infer intent. Its function is to detect statistical deviation and raise an alert accordingly.

The project consists of three principal components, each of which is addressed in turn throughout this document:

- The dataset - A synthetic file, IOT-temp-colombo.csv, constructed to provide the model with labelled examples of both normal and anomalous behaviour.
- The training notebook - IOT_Anomaly_Detection.ipynb, in which the data is processed, the model is trained and evaluated, and the final artefacts are saved.
- The live detector - anomaly_detector.py, the operational script that applies the trained model to incoming sensor readings in real time.

2. The Dataset

2.1 Reason for a Synthetic Dataset

All training data for the model originates from a single file, IOT-temp-colombo.csv. This file is synthetic in nature, meaning it was generated, rather than recorded from a physical sensor installation. This approach is not a compromise. It is the appropriate choice for the following reasons.

Training an anomaly detection model on genuine sensor logs presents two significant obstacles. First, authentic anomalies are rare events. A real sensor log may contain only a small number of anomalous observations, which is insufficient for a supervised classifier to learn the distinguishing characteristics of such events. Second, real-world logs are typically unlabelled. No ground-truth record exists identifying which readings were genuinely anomalous, and without a reliable label set, supervised learning is not feasible.

A synthetic dataset solves both problems. Because the data is generated by a controlled program, every row carries a verified label in the anomaly_detected column, providing a complete and trustworthy ground truth. Furthermore, the generator can produce anomalies in the required quantity and diversity. The fundamental distributions are modelled on the climate of Colombo, intergrating realistic temperature and humidity patterns including daytime variation, such that the patterns learned during training remain applicable to real sensor data.

2.2 Dataset Structure and Summary Statistics

The dataset spans six months, from 1 January to 1 July 2024, with observations recorded at five-minute intervals. The total number of rows is 105,351. Each row contains five columns, as described in the following table.

Column	Description	Example Value
timestamp	Date and time of the observation, recorded at five-minute intervals	2024-01-01 00:05:00
temp	Temperature in degrees Celsius	28.9
out/in	Sensor location indicator: Indoor or Outdoor	Out
humidity	Relative humidity expressed as a percentage	75.4
anomaly_detected	Ground-truth label: 0 denotes normal; 1 denotes anomalous	0

Two sensors operate in parallel, one indoors and one outdoors, resulting in approximately 52,700 observations per sensor type. The dataset contains no missing values, which eliminates the need for attribution prior to training. Under normal conditions, temperature values fall within the range of 24 to 37 degrees Celsius, and relative humidity ranges from approximately 50 to 96 percent.

2.3 Anomaly Taxonomy

Of the 105,351 total observations, 3,128 are labelled as anomalous, representing approximately 2.97 percent of the dataset. These anomalies are not homogeneous, the dataset incorporates three distinct anomaly types, as depicted in Figure 1 below.

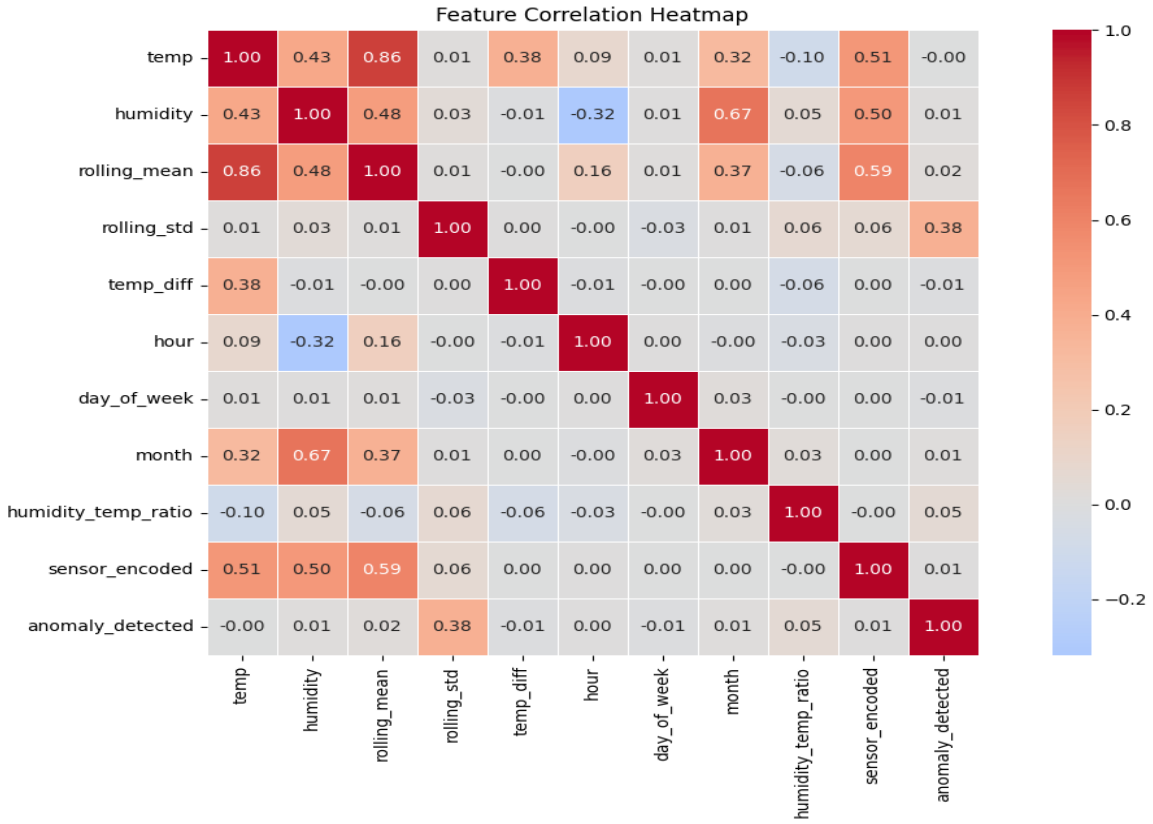

Figure 1. Six months of sensor readings. Normal observations are shown in teal, anomalous observations are shown in red. The figure illustrates high-temperature spikes, low-temperature dropouts, and contextual anomalies embedded within the normal value range.

Type 1: High-Temperature Spikes

This category comprises readings that exceed values physically attainable within the deployment environment. The dataset includes spikes reaching 67 degrees Celsius, with approximately 500 anomalous observations recorded above 40 degrees Celsius. On a real device, excursions of this magnitude are consistent with sensor hardware failure or false-data injection attacks, in which fabricated readings are deliberately submitted to the system.

Type 2: Cold Dropouts and Sensor Freezing

This category represents the inverse phenomenon, readings that collapse toward or below zero degrees Celsius. Values as low as negative 1.1 degrees Celsius appear in the dataset. In a tropical indoor environment, such readings are physically implausible and indicate power loss, disconnection, or a sensor that has become fixed on a single value. Approximately 700 anomalies fall within this cold band.

Type 3: Circumstantial Anomalies

This is the largest and most analytically challenging category, comprising approximately 1,900 anomalies. Readings in this group have temperature values that fall entirely within the normal range of 20 to 40 degrees Celsius. Their anomalous character is not apparent from the temperature value alone; it arises from contextual incongruence, such as a value that is inconsistent with the expected temperature for the time of day, or that represents an implausibly large departure from the immediately preceding reading. A reading of 39 degrees Celsius recorded indoors at 01:30 may be numerically unremarkable in isolation, yet it is contextually anomalous given the expected nocturnal conditions. Simple threshold-based rules are unable to detect anomalies of this type, which is the primary motivation for employing a machine learning classifier rather than a rule-based approach.

Significance of Anomaly Diversity

Training exclusively on high-magnitude anomalies would produce a detector that is blind to frozen sensors or contextually disguised attacks. By incorporating all three anomaly families, the dataset requires the model to learn multiple distinct warning signatures, which is what enables it to generalise to real-world, varied tampering scenarios rather than only to the most visible cases.

2.4 Class Imbalance

Anomalies constitute only 2.97 percent of the dataset, resulting in a pronounced class imbalance. This presents a well-known challenge in machine learning. A trivial classifier that assigns the label 'normal' to every observation would achieve 97 percent accuracy while failing to detect a single anomaly. This illustrates

why classification accuracy is an inadequate performance metric for imbalanced problems and why the training pipeline, described in Section 3, employs the Synthetic Minority Over-sampling Technique (SMOTE) to address the imbalance, alongside evaluation metrics that are more informative under skewed class distributions.

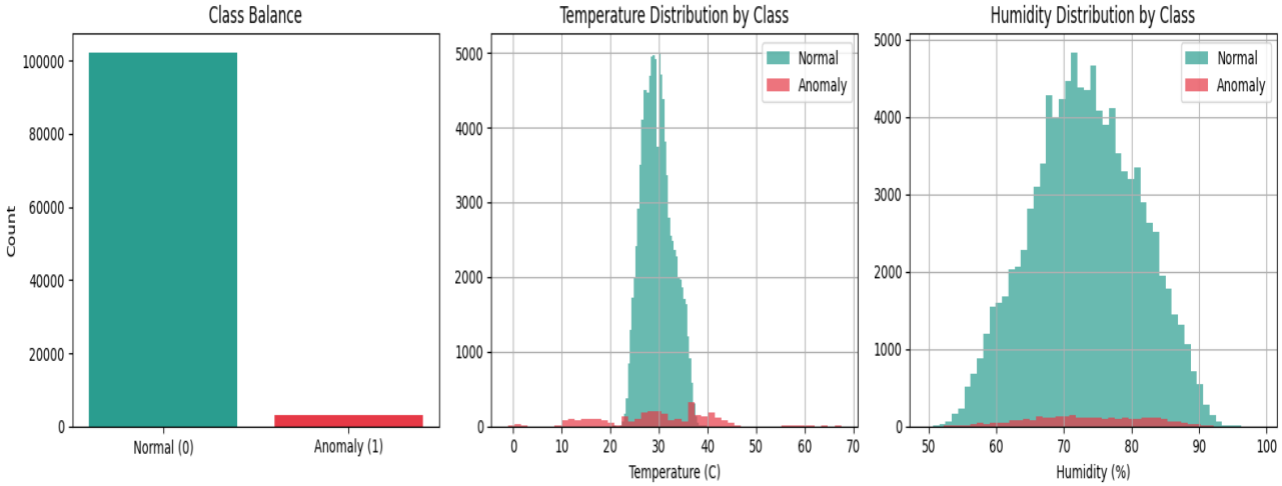

Figure 2. Left panel: class distribution illustrating the 97%/3% imbalance between normal and anomalous observations. Centre panel: temperature histograms by class, showing that anomalous readings exhibit a substantially wider distribution than normal readings. Right panel: humidity histograms by class, showing near-complete overlap, indicating that humidity has limited discriminative power for this classification task.

The centre and right panels of Figure 2 convey an important analytical observation: temperature carries substantially greater discriminative signal than humidity for this problem. This is consistent with the feature importance analysis presented in Section 3.9, where the model independently arrives at the same conclusion.

3. The Training Notebook

All model training is performed within a Jupyter notebook designated IOT_Anomaly_Detection.ipynb. The notebook is structured as a sequential pipeline of ten stages, through which raw data enters at the beginning and a fully trained, evaluated, and serialised model emerges at the end. The ten stages are summarised in the following table.

Stage	Description
1	Installation and importation of required software libraries
2	Loading the CSV dataset
3	Exploratory data analysis: shape, class balance, and distributions
4	Feature engineering: construction of derived input variables
5	Partitioning data into training, validation, and test sets
6	Correction of class imbalance via SMOTE and feature scaling
7	Training of three candidate models with hyperparameter tuning
8	Comparative evaluation on the validation set
9	Confusion matrix analysis and final evaluation on the test set
10	Serialisation of the selected model to disk

3.1 Stage 1: Library Installation and Import

The pipeline begins by installing and importing all required software libraries. Each library serves a distinct function within the pipeline.

```
!pip install imbalanced-learn xgboost --quiet


import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
    roc_auc_score, f1_score, precision_score, recall_score)
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
import joblib, os
```

The pandas and numpy libraries provide data manipulation and numerical computation functionality. The matplotlib and seaborn libraries are used for all visualisations presented in this document. The scikit-learn library supplies the machine learning models and scoring utilities. The imbalanced-learn library provides SMOTE, which is used to address the class imbalance identified in Section 2.4. The xgboost library provides one of the three candidate classifiers. The joblib library handles model serialisation.

3.2 Stages 2 and 3: Data Loading and Exploratory Analysis

The dataset is loaded and sorted chronologically by timestamp. This ordering step is a prerequisite for the rolling window features constructed in Stage 4, which depend on observations being arranged in temporal sequence.

```
df = pd.read_csv(csv_filename)
df["timestamp"] = pd.to_datetime(df["timestamp"])
df = df.sort_values("timestamp").reset_index(drop=True)

print("Shape:", df.shape)
```

Exploratory data analysis confirms that the table contains 105,351 rows and 5 columns, that no missing values are present, and that the class distribution is as follows.

```
=== Class Balance ===
              Count  Percentage
Normal (0)   102223       97.03
```

```
Anomaly (1)    3128        2.97
```

This stage also produces Figures 1 and 2 presented in Section 2. Identifying the class imbalance at this early stage ensures that appropriate corrective measures are incorporated into the pipeline before training commences.

3.3 Stage 4: Feature Engineering

The raw dataset provides temperature, humidity, time, and sensor location. These four variables are insufficient to detect contextual anomalies of the type described in Section 2.3, as contextual anomalies are defined by their incongruence with surrounding conditions rather than by their absolute values. Feature engineering addresses this limitation by constructing additional derived variables that expose temporal and statistical context to the model. This stage is the single most consequential contributor to final classification accuracy.

```
def engineer_features(frame):
    frame = frame.copy().sort_values("timestamp").reset_index(drop=True)

    # Time features
    frame["hour"]        = frame["timestamp"].dt.hour
    frame["day_of_week"] = frame["timestamp"].dt.dayofweek
    frame["month"]       = frame["timestamp"].dt.month

    # Rolling features, computed per sensor type
    frame["rolling_mean"] = (frame.groupby("out/in")["temp"]
        .transform(lambda x: x.rolling(window=10, min_periods=1).mean()))
    frame["rolling_std"]  = (frame.groupby("out/in")["temp"]
        .transform(lambda x: x.rolling(window=10, min_periods=1).std().fillna(0)))
    frame["temp_diff"]    = (frame.groupby("out/in")["temp"]
        .transform(lambda x: x.diff().fillna(0)))

    # Interaction feature
    frame["humidity_temp_ratio"] = frame["humidity"] / (frame["temp"] + 1)

    # Encode sensor type: Indoor = 0, Outdoor = 1
    frame["sensor_encoded"] = (frame["out/in"] == "Out").astype(int)
    return frame
```

The purpose of each engineered feature is summarised in the following table.

Feature	Information Provided to the Model
hour	Time of day on a 0-to-23 scale, enabling the model to learn that certain temperatures are anomalous at particular hours
day_of_week	Day of the week, encoded as an integer from 0 (Monday) to 6 (Sunday)
month	Calendar month, capturing seasonal variation across the six-month observation period
rolling_mean	Mean temperature of the preceding 10 readings, representing the recent local baseline
rolling_std	Standard deviation of the preceding 10 readings, measuring recent variability; sudden increases in this value indicate erratic sensor behaviour
temp_diff	Difference between the current reading and its immediate predecessor, detecting abrupt step changes
humidity_temp_ratio	Ratio of humidity to temperature; departures from the expected co-variation of these two variables may indicate sensor compromise
sensor_encoded	Binary indicator distinguishing indoor (0) from outdoor (1) sensor readings

Grouping by Sensor Type

The independent rolling and difference calculation performled on the indoor data stream and the outdoor data stream guarantee that the value from one sensor is only compared to the previous values of that sensor. This prevents incorrect calculations or differences from occurring between two separate sensors that would be detected by the sensors (spurious obvious discontinuities or discontinuities due to differences between manufacturers).

The beneficial aspect of calculating the rolling and difference features based on each sensor's own historical data ensures that each value is evaluated based on a minimum of variation when compared to similar historical data of the same sensor.

```
FEATURES = ["temp", "humidity", "rolling_mean", "rolling_std",
            "temp_diff", "hour", "day_of_week", "month",
            "humidity_temp_ratio", "sensor_encoded"]
```

To validate the constructed features, a correlation heatmap is computed prior to model training, as shown in Figure 3.

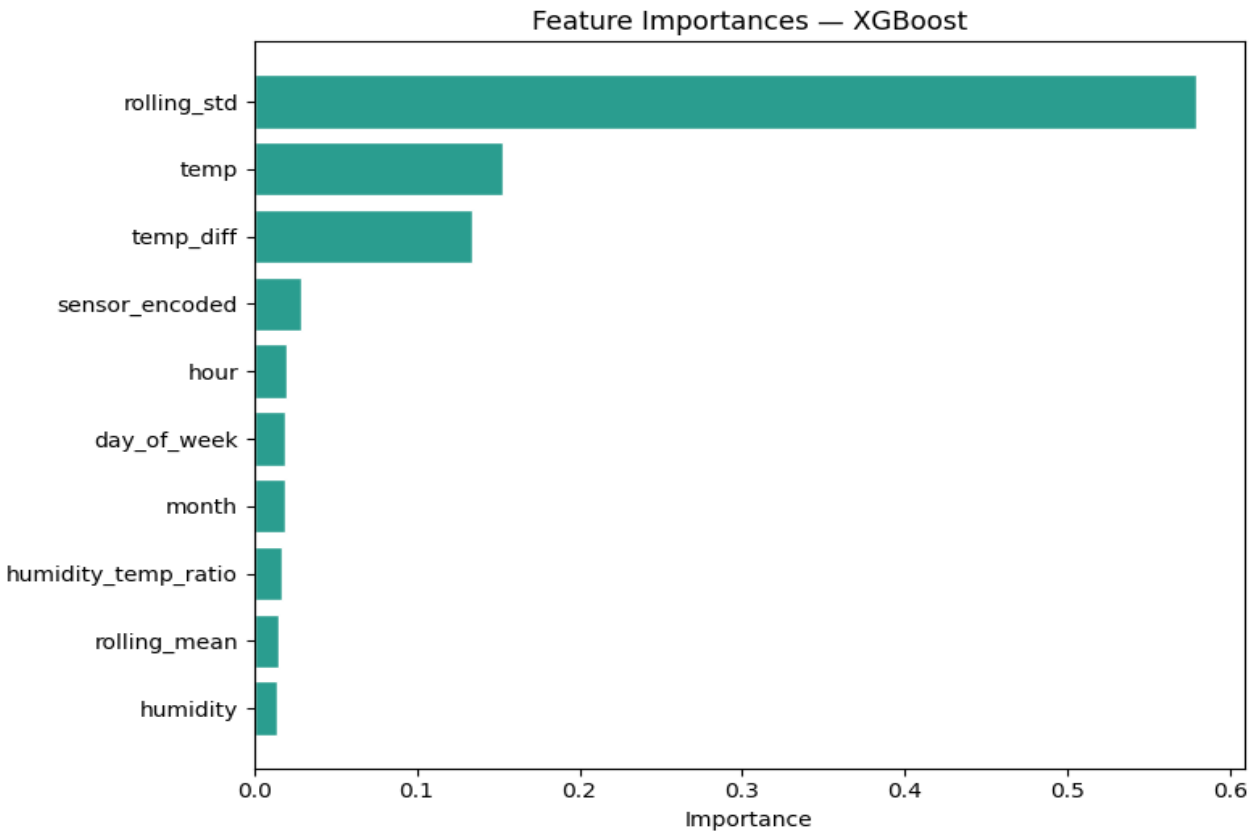

Figure 3. Feature correlation heatmap. The bottom row presents the correlation coefficient between each feature and the anomaly_detected target variable. The rolling_std feature exhibits the strongest individual correlation (0.38), consistent with expectations given its sensitivity to erratic sensor behaviour.

The correlation matrix shows that no individual feature can independently predict the anomaly label alone; therefore, a multi-feature classification model is applicable to this dataset. However, the feature that had the highest individual correlation with the target label, rolling_std (0.38), can be logically correlated with an irregular reading off a sensor and the likelihood of that sensor failing or being the victim of an attack. This was proved empirically through the feature importance analysis performed in Section 3.9.

In addition, many features exhibit a high inter-feature correlation, for example rolling.mean compared to temperature has a correlation of 0.86. This is expected, given that rolling.mean is calculated using only temperature as its basis, it does not indicate a problem of redundancy for tree-based classifiers, as was used in the analysis of this dataset.

3.4 Stage 5: Data Partitioning

Prior to any model training, the dataset is partitioned into three mutually exclusive subsets. This partitioning strategy is what ensures that the final performance estimates are unbiased.

```
# First, separate the test set (15% of total data)
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y)

# Then partition the remainder into training (70%) and validation (15%)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.176,
    random_state=42, stratify=y_train_val)
```

Subset	Proportion	Observations	Role
Training	70%	73,787	Data from which the model learns its parameters
Validation	15%	15,761	Used for model comparison and hyperparameter tuning
Test	15%	15,803	Held out until final evaluation; opened exactly once

The dataset's test set should be used only once and only as a sealed evaluation partition. It does not enter into the decision-making process as part of developing or selecting models. Instead, the only time you check your performance is at the end of the process (see Section 3.8). This allows you to be sure that you are reporting your test performance based on your models' true ability to generalise, and not because of something you did in optimisation.

There are two aspects of the splitting process that merit particular attention: the stratify=y argument and the random_state=42 argument. The stratify=y argument keeps each partition from being significantly different in terms of the % of anomalies from the complete dataset (2.97%). Thus, this prevents subsets from being inadvertently easier/harder to classify than other subsets. The random_state=42 argument locks down the random number generator, making the splitting process deterministic and repeatable each time the notebook is run independently.

3.5 Stage 6: Class Imbalance Correction and Feature Scaling

After splitting the data into training & test sets, the imbalance in class as identified in Section 2.4 was resolved by using SMOTE to "smooth" out the training set only. SMOTE synthesises new minority-class observations by creating a new sample of data points based on interpolation between existing samples of data rather than copying from existing data. By using this method, the classification algorithm receives many examples of how anomalies are related but has the same representation of anomalies as its training dataset.

```
smote = SMOTE(random_state=42, k_neighbors=5)
X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

Before SMOTE:   Normal 71,596  |  Anomaly  2,191
After  SMOTE:   Normal 71,596  |  Anomaly 71,596
```

After applying SMOTE (Synthetic Minority Over-Sampling Technique), the Training Dataset increases from about 73800 to 143192 samples, with a perfectly balanced set of inputs. Validation and Testing Samples are not adjusted so they can accurately reflect how classes would be represented in the operational environment.

The last step at this stage is Feature Scaling; each feature is sent through a normalisation process to have a mean of 0 and a standard deviation of 1, which allows features to exert equal influence over the model without being elevated because of their numerical scale alone.

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train_sm)   # learn parameters and apply
X_val_sc   = scaler.transform(X_val)            # apply saved parameters only
X_test_sc  = scaler.transform(X_test)           # apply saved parameters only
```

Importance of Consistent Scaling

The parameters for the Normalisation are only calculated based on the Training data and then applied directly to the Validation and Testing sets using the same Normalisation parameters each time. The Normalisation object that was created at the end of the previous stage is saved to an external file name (scaler. pkl) and used each time the Live Detector receives new data points. Failure to use the same Normalisation parameters creates skewed Feature Distributions and causes all future predictions to be adversely affected since the model was trained with the original Normalisation parameters..

3.6 Stage 7: Model Training and Hyperparameter Configuration

Three candidate classifiers are trained in parallel, allowing a data-driven selection of the most appropriate model for this task.

```
models = {
  "Logistic Regression": LogisticRegression(
      class_weight="balanced", max_iter=1000, random_state=42),

  "Random Forest": RandomForestClassifier(
      n_estimators=200, class_weight="balanced",
      max_depth=15, min_samples_leaf=5,
      random_state=42, n_jobs=-1),

  "XGBoost": XGBClassifier(
      n_estimators=200, max_depth=6, learning_rate=0.1,
      scale_pos_weight=(y_train_sm==0).sum()/(y_train_sm==1).sum(),
      eval_metric="logloss", random_state=42, n_jobs=-1),
}
```

The three models and the rationale for including each are described in the following table.

Model	Description	Role in Comparison
Logistic Regression	A linear classifier that partitions the feature space with a single hyperplane	Baseline comparator, a more complex model that does not outperform this baseline offers insufficient justification for its added complexity
Random Forest	An ensemble of decision trees that combines predictions through majority voting	A robust non-linear classifier representing the class of bagging ensemble methods
XGBoost	A gradient-boosted tree ensemble in which each successive tree corrects the errors of its predecessors	A boosting ensemble that typically achieves strong performance on structured tabular data

The key hyperparameter choices are explained below.

- n_estimators=200: Constructs 200 trees per ensemble. A larger ensemble generally reduces variance at the cost of increased computation time.
- max_depth (15 for Random Forest, 6 for XGBoost): Restricts tree depth to prevent overfitting by limiting the model's ability to memorise individual training observations.
- learning_rate=0.1: Controls the magnitude of each correction step in XGBoost. Smaller values lead to more conservative and typically more accurate convergence.
- min_samples_leaf=5: Requires that each terminal leaf node in the Random Forest be supported by a minimum of five observations, preventing the model from constructing rules around isolated outliers.
- class_weight='balanced' and scale_pos_weight: Secondary imbalance corrections applied at the model level, instructing the classifiers to assign greater loss to misclassified minority-class observations.

Regularisation and Generalisation

Most of the hyperparameters above act as regularisation methods to limit how much capacity (overfitting) the model has so that it can better learn from the training data and generalise to observations that it has not seen previously. In Stage 5, the held-out test will be used to definitively measure if this regularisation was successful.

3.7 Stage 8: Comparative Evaluation on the Validation Set

All of the models are evaluated against a validation dataset (i.e., data not used in training) and contain different evaluation metrics; standard classification accuracy is useless because of class imbalance discussed in section 2.4 above.

- Precision: the percentage of anomalous observations predicted by the model are truly anomalous. A low level of precision suggests that the model produces too many false positives.
- Recall (also called sensitivity): the percentage of truly anomalous observations caught by the model. Recall is the most critical metric for a safety monitoring system, as missing an anomaly would lead to a severe malfunction.
- F1 Score: the harmonic mean of precision and recall. This metric provides a single statistic that considers both metrics equally. The F1 Score is used for the primary criterion of model selection.
- Receiver Operating Characteristic - Area Under Curve (ROC-AUC): a measure of the model's ability to separate two classes across all thresholds at the time point of evaluation. A ROC-AUC of 0.5 is indicative of random performance while a ROC-AUC of 1.0 is perfect separation.

The validation set results for all three models are presented in the following table.

Model	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.208	0.808	0.330	0.932
Random Forest	0.640	0.940	0.761	0.997
XGBoost (selected)	**0.698**	**0.915**	**0.792**	**0.996**

Logistic Regression has acceptable recall (0.808), but poor precision (0.208) which means that almost four out of five alarms generated by this model will be false positives. This number of false alarms makes it impractical for operations to use. Both Random Forest and XGBoost achieve better performance metrics than Logistic Regression, with the highest F1 score of 0.792 being achieved by XGBoost, thereby selecting it as the deployed model. The same metrics in graphic form appear in Figure 4.

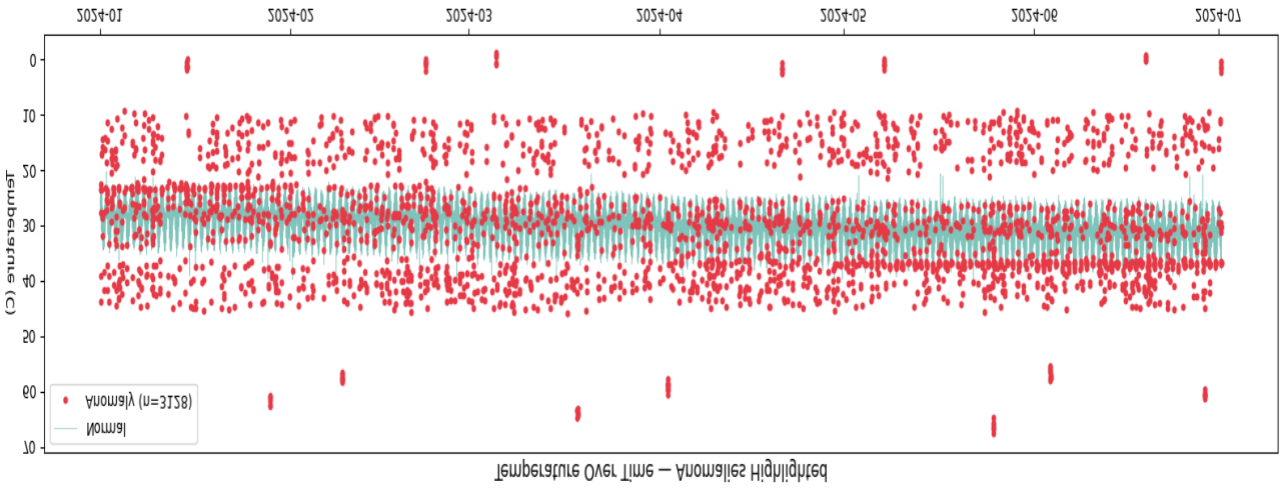

Figure 4. Left panel: ROC curves for all three models on the validation set. Random Forest and XGBoost approximate the ideal top-left corner, while Logistic Regression shows comparatively reduced discriminative ability. Right panel: grouped bar chart comparing all four evaluation metrics across the three models.

3.8 Stage 9: Confusion Matrix Analysis and Final Test Evaluation

A confusion matrix provides a complete and unambiguous account of a model's classification errors by tabulating outcomes across four categories.

- True Negative: A normal observation correctly classified as normal.
- True Positive: An anomalous observation correctly identified as anomalous.
- False Positive: A normal observation incorrectly classified as anomalous, constituting a false alarm.

- False Negative: An anomalous observation that the model failed to detect. This is the error type of greatest concern in a safety monitoring context.

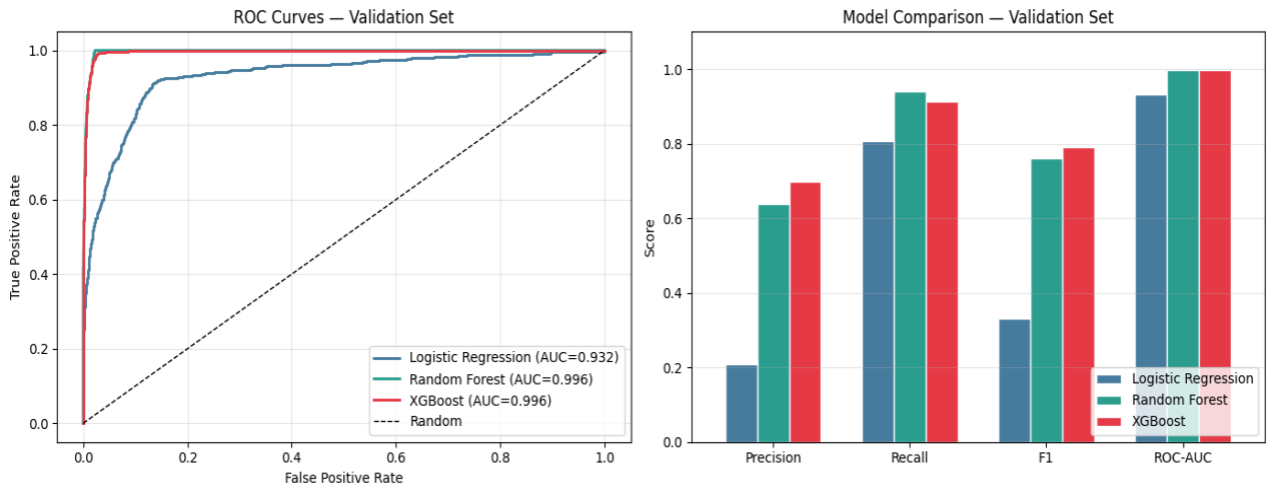

Figure 5. Confusion matrices for all three models on the validation set. XGBoost (right) misses 40 of 468 anomalies while generating 185 false alarms, representing the most favourable balance between the two error types.

The confusion matrix counts for the validation set are presented numerically in the following table for precise comparison.

Model	True Negatives	False Positives	False Negatives	True Positives
Logistic Regression	13,850	1,443	90	378
Random Forest	15,045	248	28	440
XGBoost (selected)	15,108	185	40	428

Although Logistic Regression has generated an unmanageable number of false alarms (1,443) in an operational environment, Random Forest produces the fewest total missed anomalies (28) and the fewest total false alarms (185 compared with 248) at the expense of only a small number of missed anomalies (40 compared with 28). These relative performance metrics justify the selection of XGBoost as the preferred model because it produced superior overall performance metrics than the other two models, including the highest F1 score. During the selection of XGBoost as the preferred model, the performance of the test set was assessed for the first and only time. No adjustments can be made to the model after reviewing its performance

with the test set. The performance of both the validation and test sets will produce a confusion matrix, which is shown in Figure 6.

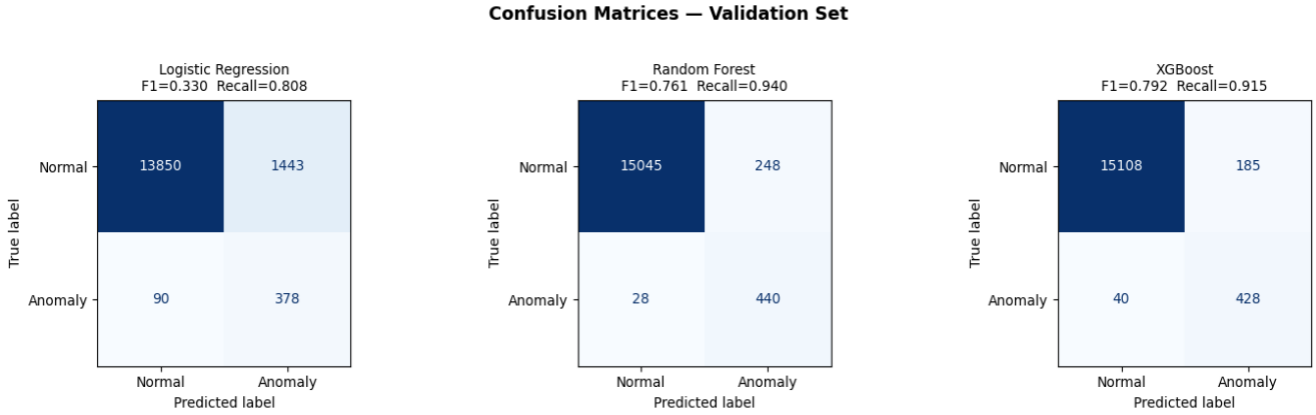

Figure 6. Confusion matrix for the XGBoost model on the held-out test set (15,803 observations). The model correctly identifies 424 of 469 genuine anomalies, misses 45, and generates 173 false alarms.

Outcome	Count	Interpretation
True Negative	15,161	Normal reading correctly cleared; the predominant outcome in routine operation
False Positive	173	Normal reading incorrectly flagged as anomalous; a manageable false alarm rate
False Negative	45	Anomalous reading missed by the model; this is the consequential error type and is kept as low as possible
True Positive	424	Anomalous reading correctly detected and reported

The model achieves a recall of approximately 90 percent (424 of 469 anomalies detected) and a precision of approximately 71 percent (424 of 597 alerts verified as genuine) on the test set. The final test scores are presented in full below.

Metric	Value	Interpretation

F1 Score	0.80	A strong balance between anomaly detection and false alarm control
ROC-AUC	0.997	Near-perfect discriminative ability between normal and anomalous classes
Recall (anomaly class)	0.90	Approximately nine in ten genuine anomalies are detected
Precision (anomaly class)	0.71	Approximately seven in ten raised alarms correspond to genuine anomalies

The close agreement in performance of both the validation and test sets indicates the model has successfully generalised to unseen cases. Strong correlation between final and practice exam scores indicates the model has extracted transferable patterns, as opposed to the model having merely memorised training data artefacts.

3.9 Feature Importance Analysis

Due to the ensemble nature of XGBoost (a tree-based ensemble), one can examine which of the input features contributes to predictions made by the model. This allows for an interpretability check of the model: if the features that are most heavily relied upon by the model would also be considered by a domain expert to be the most valuable features, this suggests that the model has learned meaningful patterns.

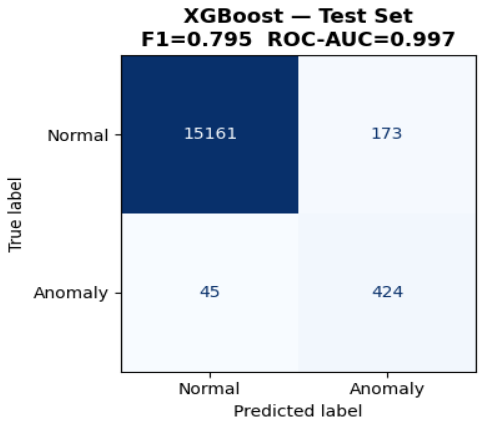

Figure 7. Feature importance rankings for the XGBoost model. The rolling_std feature accounts for 0.58 of the model's total decision weight, substantially exceeding all other features.

Rank	Feature	Importance	Model Utilisation
1	rolling_std	0.58	Recent variability in sensor readings; the dominant discriminative signal by a substantial margin
2	temp	0.15	The raw temperature value of the reading
3	temp_diff	0.13	The magnitude of the step change from the preceding reading
4	sensor_encoded	0.03	Indoor versus outdoor sensor indicator
5-7	hour, day_of_week, month	~0.02 each	Temporal context features
8-10	humidity_temp_ratio, rolling_mean, humidity	~0.01 each	Secondary supporting signals

The top three features cumulatively account for approximately 86% of the model's decision-making weight and are all related to temperature, as opposed to humidity; this finding matches the distributional results presented in Section 2.4 and the correlation analysis presented in Section 3.3.

The theoretical advantage of having rolling_std as the highest-ranking feature is clear; normal operating behaviour for sensors has stable, smoothly changing readings, while erratic sensor values (whether as a result of equipment failure or external interference) will result in increased variability. The model independently identified rolling_std as being the most informative feature, which supports the physical reasoning that there is a true, meaningful relationship to the variance of readings as opposed to spurious correlations.

3.10 Stage 10: Model Serialisation

The final stage of the pipeline writes the trained model and its associated artefacts to disk for use by the live detection system.

```
joblib.dump(best_model, "models/best_model.pkl")
joblib.dump(scaler,     "models/scaler.pkl")

metadata = { "model_name": best_name, "features": FEATURES,
             "val_f1": ..., "test_f1": ... }
json.dump(metadata, open("models/metadata.json", "w"), indent=2)
```

File	Contents
best_model.pkl	The serialised XGBoost model; the primary classification artefact
scaler.pkl	The fitted StandardScaler from Stage 6; must be applied to all new observations prior to classification
metadata.json	A record of the model name, input feature list, and validation and test performance scores

Note on metadata.json Consistency

The test F1 score achieved by the XGBoost model generated by the training run covered in this document will be around 0.80. The metadata.json file produced by a deployed model will not have either a different model name or a perfect score of 1.0 if the file is left over from a previous or different model build; hence, the deployment would not be accurately described by the file. The metadata.json file should be regenerated from the same training run as that in which the live model files were generated (produced) in order to avoid the possibility of confusion. Any F1 score reported as being 1.0 should be viewed with suspicion, as 1.0 is an uncommon result for real-world classification problems.

4. Decision Thresholds

The XGBoost classifier does not classify an observation as belonging to a given class label; rather it provides a probability of how likely it is that an observation is an anomaly. A binary classification is required to derive a discrete class label from the XGBoost classifier output, which is in the form of a continuous scale index of the likelihood of being an anomaly. To provide a binary classification, the system implements two independent, complementary operating thresholds.

4.1 Probabilistic Model Threshold

The model has a default classification threshold of 0.5. If the model assigns more than a 50 percent likelihood of anomaly to any observation, it will classify that observation as anomalous. In addition to classifying an observation as anomalous or not, the live detector will also provide the operator with the actual anomaly probability value. This will allow the operator to differentiate between high-confidence detections, and detections that fall just above or below the classification threshold. All validation and test performance metrics, as provided in Section 3, are based off of this classification threshold.

Threshold Adjustment

The classification threshold of 0.5 is a reasonable starting point, but may be modified. For example, if the threshold is decreased to 0.3, there would be an increase in the number of true positive detections, but also an increase in the number of false alarms. Conversely, if the threshold is increased, there would then be an increase in the number of false negatives, but a decrease in the number of false alarms. Since the live detector provides users with the raw anomaly probability when it makes a classification decision, end users will have the ability to adjust the classification threshold based on their operational experience and will not need to retrain the model to accomplish this.

4.2 Physical Safety Rule Threshold

As an additional source of safety that operates separately from the machine learning classification, the live detector provides hard physical limits on temperatures measured by the live detector.

```
TEMP_BOUNDS = {
    "indoor":  (22, 35),
    "outdoor": (22, 38),
}
```

This rule offers assurance that there are rules in place to avoid Type 1 or Type 2 Anomaly data inconsistencies or defects specified in part two (Section 2.3). The trained machine learning model combined with the defined physical world rules represents complementary source of potential failure; where the model

handles issues of contextual anomalies that are often very small in magnitude and where machine learning statistical techniques are ineffective at making judgments; whereas physical world rules provide firm procedural rules against both very large de facto anomalies and impossible readings that may or may not be detectable by the training model.

5. The Live Detector

Section 3 describes an offline training pipeline that operates on a static set of data over six months. The software tool Anomaly Detector ("anomaly detector.py") operates in real-time mode on the "Sentinel Guard" Server and receives real time ESP32 data and provides a reaction based on what the model was trained to do as opposed to an operation of stand-alone application.

The difference between operating at the original data level and correctly responding to incoming data in real-time is significant between the static environment of the training notebook, which had access to the full set of data and stored live operational level data in memory for the past six months, versus the operational environment where data arrives one at a time and the system must extract the features that had been engineered into the trained model. The subsequent sections present a description of the functionality of the live detector.

5.1 Configuration Parameters

```
MODEL_DIR   = "models"
WINDOW_SIZE = 10      # must match the value used during training

FEATURES = ["temp", "humidity", "rolling_mean", "rolling_std",
            "temp_diff", "hour", "day_of_week", "month",
            "humidity_temp_ratio", "sensor_encoded"]

TEMP_BOUNDS = { "indoor": (22, 35), "outdoor": (22, 38) }

HUMIDITY_FALLBACK = { 1:72, 2:70, 3:74, 4:79, 5:83, 6:85,
                      7:83, 8:83, 9:83, 10:82, 11:79, 12:76 }
```

The FEATURES list indicates the 10 model inputs in the exact order as the classifier was trained to receive them; any differences in that order would create a prediction error without an error that occurs at runtime. Using the same WINDOW_SIZE value as the window parameter used for training is important as the rolling features created by the detector are only meaningful to the model if they were created over the same length of time as the original window used to create them. The HUMIDITY_FALLBACK dictionary contains statistically valid average monthly relative humidity values for the months of the year in Colombo, and provide a last resort for when sensor data is not available.

5.2 Humidity Sensor Fault Handling

The DHT22 temperature/humidity sensor used in the ESP32 Board has a higher failure rate than the temperature sensor. Therefore, whenever the humidity sensor fails the humidity value returned will be `null`. If an incoming value of `null` is not handled appropriately, this will be passed to the calculation of the humidity/temperature ratio, thereby rendering the humidity/temperature ratio and all other predictions thereafter as `invalid`. To eliminate the chance of this happening, the detector has a 3-tiered fallback mechanism.

```
_last_known_humidity = { "indoor": None, "outdoor": None }

def _resolve_humidity(key, humidity):
    if humidity is not None:
        _last_known_humidity[key] = float(humidity)
        return float(humidity), "live"

    if _last_known_humidity[key] is not None:
        return _last_known_humidity[key], "last_known"

    month = datetime.now().month
    base  = HUMIDITY_FALLBACK.get(month, 78)
    value = float(base if key == "outdoor" else base - 8)
    return value, "fallback"
```

The Humidity function guarantees that numeric humidity values will always be provided to the feature construction process. The three levels of fallback in decreasing order of preference are as follows. First, if a live reading is available, that reading is read and stored into memory. The second option is if there is no live reading available for the sensor and, a valid historical reading is available for that particular sensor, then the last historical reading will revert to being used. Finally, if the sensor never produced a valid reading during the current session, then the climatological monthly mean for the Colombo region will be used instead. A string describing the source of the reading will also be returned so that downstream consumers can identify whether live or estimated readings were made.

5.3 Real-Time Feature Reconstruction

For rolling window features which calculate rolling_mean and rolling_std, a temporal history of preceding readings is required. In the training notebook, this temporal history is accessible implicitly through the

complete data set. Conversely, in 'live' mode, the detector must have and keep an explicit running history of the most recent readings from each of the sensors individually.

```
_history = {
    "indoor":  deque(maxlen=500),
    "outdoor": deque(maxlen=500),
}
```

The most recent readings (500) from the sensors are stored in a double-ended queue that has maxlen=500. The oldest of the current 500 readings is automatically discarded when a new 501st reading is received. Thus, providing enough history for the 10-reading rolling window while also limiting the amount of memory used.

Each new reading is added to the appropriate sensor history, and the complete feature set is then recomputed from the collected record.

```
def _build_features(key, now, temp, humidity_resolved):
    _history[key].append({
        "time": now, "temp": float(temp),
        "humidity": humidity_resolved,
    })
    df = pd.DataFrame(list(_history[key]))

    df["hour"]        = df["time"].dt.hour
    df["day_of_week"] = df["time"].dt.dayofweek
    df["month"]       = df["time"].dt.month
    df["rolling_mean"] = df["temp"].rolling(WINDOW_SIZE, min_periods=1).mean()
    df["rolling_std"]  = df["temp"].rolling(WINDOW_SIZE,
min_periods=1).std().fillna(0)
    df["temp_diff"]    = df["temp"].diff().fillna(0)
    df["humidity_temp_ratio"] = df["humidity"] / (df["temp"] + 1)
    df["sensor_encoded"]      = 1 if key == "outdoor" else 0

    row = df.iloc[[-1]][FEATURES]
    if row.isna().any().any():
        row = row.fillna(0)
    return row.values
```

This function mimics exactly the feature engineering logic contained in Stage 4 of the notebook and will format the incoming readings to be placed into the model in the same manner as the training observations. The only observed row is the last row (the one containing the reading that is currently being calculated) from df.iloc[[-1]]. The closing fillna(0) operation replaces any empty values back with a zero value so that this function will fail gracefully and will not produce an exception on unexpected data input.

5.4 The Prediction Function

The predict function is the primary interface through which the server obtains anomaly classifications. It integrates all components described in the preceding sections.

```
def predict(temperature, humidity, sensor_label="Indoor"):
    if _model is None or _scaler is None:
        return { "anomaly_detected": 0, ...,
                 "error": "Model not loaded" }

    key = sensor_label.strip().lower()
    now = datetime.now()

    humidity_resolved, humidity_source = _resolve_humidity(key, humidity)
    feature_row = _build_features(key, now, float(temperature),
                                  humidity_resolved)
    X_scaled    = _scaler.transform(feature_row)

    prediction = int(_model.predict(X_scaled)[0])
    proba      = float(_model.predict_proba(X_scaled)[0][1])

    low, high    = TEMP_BOUNDS.get(key, (22, 38))
    out_of_range = float(temperature) < low or float(temperature) > high
    if out_of_range:
        prediction = 1

    return { "anomaly_detected": prediction,
             "anomaly_proba":    round(proba, 4),
             "out_of_range":     out_of_range,
             "humidity_used":    round(humidity_resolved, 1),
             "humidity_source":  humidity_source }
```

The function runs through the following sequence of steps for every new reading. Firstly, it will check whether the model artefacts have been successfully loaded; if not, it will return a clearly defined error response (and not throw an unhandled exception). Secondly, it will resolve the humidity value (i.e. via the fallback method contained in Section 5.2). Thirdly, it will build the full feature vector from the sensor history. Fourthly, it will scale the feature values using the saved scaler. Fifthly, it will get a binary classification and a probability from the model using the scaled features. Finally, it will produce an anomalous classification for the reading if the temperature is outside its limits.

Critical Dependency on Consistent Scaling

To optimise the performance of the anomaly detection model, the training data is prepared using a standardised format. This ensures that any errors or inconsistencies in the original data will not contribute to future performance degradation. As a result, the two data artefacts (scaler.pkl and best_model.pkl) can be used together when present in the same directory.

6. Integration with the Server

The anomaly detection module is part of a larger server application (Server_file.py). While this document does not provide a complete description of the server architecture, understanding where the anomaly detection module fits into the overall design constraints of the subject system will help you further contextualise its design constraints. Incoming packets from the ESP32 device must pass through a number of security checks

before being sent to the anomaly detection module. These security checks are done in a linear fashion meaning that the sensor values are only extracted from a packet and sent to the anomaly detection module if that packet passes all security checks.

- Flood protection: Packets arriving too quickly (not consistent with typical rates observed for a given sensor) will be considered a denial-of-service attack and will be temporarily suppressed.
- Integrity and authenticity verification: Each packet includes a cryptographic signature (HMAC) as well as encrypted payload (AES) and therefore packets that do not posses these characteristics will be treated as forged/corrupted and isolated from processing.
- Replay protection: Each packet has a sequence number and those which are checked for existence prior to processing are dropped as potential replay attacks.
- Physical range validation: Any packets that contain sensor readings that could not physically occur from functioning sensors are isolated prior to being sent to the ML module.

Only packets that satisfy all of the above criteria are decrypted and their sensor values passed to the anomaly detector.

```
anomaly_result = anomaly_detector.predict(
    temperature=temperature,
    humidity=raw_humidity,
    sensor_label=SENSOR_LABEL)
```

A detector produces a result that can be used in three different ways at the same time: as real-time information, written to a server console for monitoring by an operator; stored in the cloud database (Firebase) along with the sensor reading; and logged as an event in the event database when abnormal behaviour is detected so that alert is sent to predefined groups. In addition to the above, the server has a resilience capability, which allows the server to operate with some basic level of anomaly detection, if the model artefacts are not able to be loaded at the start-up, through a fallback measurement based on the physical range defined from Section 4.2.

Layered Detection Architecture

The server employs a defence-in-depth approach whereby each layer of protection addresses a unique class of threat (multiple classes of threats). Fixed rule-based threats can be dealt with by means of cryptographic

controls and rate limiting; statistical reasoning (through Machine Learning) can be used to deal with context-based threats; and in conjunction with deterministic rules, provide a final level of protection against any behaviour that is outside the established parameters. The overall system, therefore, provides a greater degree of reliability than any single mechanism.

7. Summary

This document has presented a complete account of the ML Model Classification system for IoT sensor anomaly detection. A summary of Principal Contributions of Each Phase of the Project is as follows:

- Dataset Construction: A Synthetic Fully Labeled Dataset of Colombo Temperature and Humidity Observations for a 6-month Period was Produced; 6 Anomaly Types Were Included in the Dataset to Make Sure There Were Enough Examples to Learn from for Training.
- Feature Engineering: 10 New Feature Variables Derived from Raw Observations Were Added to Capture Temporal, Local, and Interrelated Variability, Allowing for the Detection of Contextual Anomalies That Cannot Be Detected by Threshold Based Systems.
- Rigorous Training Methodology: 3 Different Sets of Data Were Split into Training, Validation and Testing; Class Corrections for Class Imbalance Were Done Using SMOTE on the Training Data; Training Example Models Were Compared Using Standard Metrics for Evaluating Imbalanced Classification Models.
- Model Selection and Validation: XGBoost Was Selected for Use as the Best Model Based on an F1 Score; On a Held Out Test Set XGBoost Achieved Recall of Approximately 90% and F1 Score of 0.80; Test Performance Was Similar to Validation Test Results Showing True Generalisation capability for the XGBoost Model.
- Operational Deployment: The Anomaly Detector Module generate Data Features from Rolling Historical Data Using Sensor Fault Handling Techniques and Evaluates Each Incoming Sensor Reading Using The Probability Model and The Physical Safety Rule; For the Anomaly Detector To Determine If Each Sensor Reading Contains An Anomaly.
- Layered server integration: Each layer of the server architecture offers a separate protection layer to its respective attacks from that type. This layered approach is very significant because it builds upon the previous layer to enhance the overall level of security for the final system when entirely constructed.

The result is a system that far exceeds what would be expected from traditional threshold-based monitoring. Every incoming sensor value is evaluated considering the statistical context from the sensor's recent history; therefore, the system can conduct an evaluation of each incoming sensor value concerning the level of predictability associated with that incoming sensor value for an operational environment being considered. This is what gives a machine learning-based detector its significant difference from a rules-based machine learning interceptor..